



EVSS: An efficient verifiable search scheme over encrypted cloud data

Qian Zhou¹ · Hua Dai^{2,3} · Wenjie Sheng² · Yuanlong Liu² · Geng Yang^{2,3}

Received: 11 July 2022 / Revised: 7 August 2022 / Accepted: 20 August 2022 /

Published online: 14 September 2022

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2022

Abstract

Cloud computing is now being used by more and more enterprises and individuals. To protect the privacy of outsourced data in the cloud, the searchable symmetric encryption is adopted. However, verifying search results to detect whether there is malicious behavior in the cloud server is still a challenge. In order to overcome this problem, we propose an efficient verifiable search scheme (EVSS), which allows users to retrieve ciphertext and verify the search result efficiently. A novel RSA accumulator-based Patricia tree (RAP-tree) is designed as the search index, where the token of a keyword and the proof evidence of documents containing the keyword are taken as the key-value pair to be placed in a node of the tree. By using the RAP-tree index, the efficient verifiable keyword search over encrypted data is realized. The security of the proposed scheme is analyzed by theoretical proofs. Experimental results on real data sets show that the proposed scheme is efficient and effective in terms of search processing and result verification.

Keywords Cloud computing · Searchable symmetric encryption · Verifiable search · RSA accumulator

✉ Hua Dai
daihua@njupt.edu.cn

Qian Zhou
zhouqian@njupt.edu.cn

Wenjie Sheng
1219043731@njupt.edu.cn

Yuanlong Liu
1021041407@njupt.edu.cn

Geng Yang
yangg@njupt.edu.cn

¹ School of Modern Posts, Nanjing University of Posts and Telecommunications, Nanjing 210023, China

² School of Computer Science, Nanjing University of Posts and Telecommunications, Nanjing 210023, China

³ Jiangsu Key Laboratory of Big Data Security and Intelligent Processing, Nanjing 210023, China

1 Introduction

As a successful business model, cloud computing has developed rapidly in recent years. Its advantages such as charging on demand, high scalability and convenient access are favored by many enterprises and individuals. More and more users choose to upload data resources or private files to cloud, and the scale of data stored in cloud server is also increasing. In order to prevent the privacy of the data from being leaked, users encrypt the data before outsourcing it. Searchable symmetric encryption (SSE) is proposed to improve the availability of data, and has gradually become a mature scheme after years of development.

In recent years, many searchable symmetric encryption schemes [1–8] have been proposed. However, most of the schemes are based on the honest-but-curious threat model, where the cloud server is assumed to perform the search processing honestly but it is curious of obtaining sensitive information from users' data. In these schemes, the user trusts the search results returned by the cloud server. However, in practical, the cloud server may not be trustworthy. On one hand, the cloud server storing a large amount of data is attractive and vulnerable to external attacks, and the stored data may be tampered or falsified. On the other hand, in order to save the cost of computation, the cloud server may perform dishonest retrieval and return incomplete results. In order to avoid the losses caused by dishonest behavior of the cloud server, some studies have been proposed to verify the completeness of search results, which are the verifiable SSE schemes. The schemes [9–11] either need a lot of storage space, or the search efficiency is low. Besides, the schemes based on Trie tree [12, 13], Merkle hash tree [14] and RSA accumulator [15] are proposed to improve the search efficiency, but they still cannot achieve a good balance between space cost and time cost. Therefore, it is necessary to study efficient verifiable search schemes to promote the development of cloud computing.

Based on above requirements, this paper proposes an Efficient Verifiable Search Scheme (EVSS), which allows the user to conduct efficient and verifiable search on the ciphertext set based on keywords. Our idea is to build a RSA accumulator-based Patricia tree (RAP-tree) index, and the novel index is used to perform search processing and result verification. In our scheme, a prime number is computed for each keyword-ciphertext pair, and then these prime numbers are added to RSA accumulator through modular exponentiation to generate local evidence. After calculating the proof for each keyword, a RAP-tree index is built, and the index is outsourced to the cloud. If the cloud server receives a search trapdoor, it will perform search operation and return search results and proof. The RSA accumulator will be reconstructed by the user to verify the integrity of search results. The EVSS scheme with the RAP-tree index can achieve efficient search, each search need no additional computing resources to generate proof, and the additional communication cost of proof is very small by using RSA accumulator to verify.

The main contributions of this paper are summarized as follows:

- We propose an EVSS scheme to achieve an efficient verifiable search on ciphertext in cloud environment.
- We propose and construct a RAP-tree, which enables fast retrieval of keywords and efficient verification of search results.
- We analyze the security of the EVSS scheme in theory, and analyze the performance of the EVSS scheme through experiments on real database.

Paper organization In Section 2, we discuss some existing related work, and in Section 3 we introduce some preliminary knowledge. In Section 4, we list the necessary parameters and the definition of the scheme. Section 5 describes the concrete implementation of the scheme. In Section 6, we carry out the security analysis. We give the experimental results and analysis in Section 7. Finally, we present our conclusions in Section 8.

2 Related work

2.1 Searchable symmetric encryption

Searchable Symmetric Encryption (SSE) was first proposed by Song et al. [3]. They encrypt each keyword in the document and scan the whole database in order to realize the search. Goh [4] proposes a scheme based on Bloom filter and gives the security definition of SSE. However, due to the false positive probability of Bloom filter, the cloud server may return documents that do not contain search keywords. Based on inverted index, Curtmola et al. [5] proposed two different schemes: SSE-1 and SSE-2. SSE-1 is secure against non-adaptive chosen-keyword attack (CKA1), while SSE-2 is secure against adaptive chosen-keyword attack (CKA2). Compared with SSE-1, SSE-2 has higher security. Similarly, based on the "honest and curious" model of cloud server, different SSE schemes are proposed in [6–8, 16–23]. These early works have realized keyword retrieval on encrypted data sets, but they have not considered the problem of compiling encrypted data in cloud server, which can not guarantee the availability of search results. At the same time, these schemes can not meet the increasing requirements in time efficiency or space efficiency.

2.2 Verifiable searchable symmetric encryption

With the development of SSE scheme, more and more researchers begin to realize the importance of verifiability of search results, which leads to a lot of research on verifiable searchable symmetric encryption (VSSE) scheme, and there are many corresponding schemes. Kurosawa et al. [9] proposed a UC-secure scheme to detect the dishonesty of the cloud server. This scheme has better security than SSE-2, but needs a linear search time. The linked signature chaining scheme is proposed in [10, 11] where the data owner signs for consecutive data items to form a linked chaining, and any modification of data will lead to the complete change of the chaining, so the dishonest behavior of cloud server is easy to be detected. However, the linked signature chaining obviously leads to very high computational costs, storage overhead, and user-side verification time [24]. Chai et al. [12] proposed a scheme based on Trie tree, which has faster search speed, and Wang et al. [13] improved on this basis to reduce the types of elements stored in the node. However, both of the above two schemes need a lot of space to store the tree index. [25] proposes a verifiable data structure based on multiset hash function, which ensures forward security and realizes effective verifiable data update. [26] realizes the verification of the relationship of query keywords in the file by using the Relationship Authentication Label (RAL), and can generate audit certificates without exposing sensitive information. However, this scheme requires Third Party Auditor to participate in the search process. The Merkle hash tree [14, 27, 28] is used to verify data elements in large database. By adding data elements to the leaf node of the tree, the tree structure is constructed layer by layer from the leaf node to

the root node, and finally an unique root node is obtained. The change of any element in the data set would lead to the change of the root node. The Merkle Patricia Tree is proposed in GSSE [29] to reduce the storage cost of index structure in Merkle hash tree-based scheme. It reduces the storage space by reducing the depth of the tree. However, in the above two schemes, the proof scale provided by cloud server to DU is large, which brings more communication overhead. RSA accumulator [15, 30–32] can aggregate a large number of data sets into a fixed size value to realize membership verification, which can effectively reduce the communication cost. Applying RSA accumulator to compressed prefix tree structure can realize the effective combination of efficient retrieval and result verification.

3 Preliminary

3.1 RSA accumulator

In this paper, we use the RSA accumulator [15] to verify whether the ciphertext returned by the cloud server is correct and belongs to the ciphertext set of the data owner. RSA accumulator can provide a constant size representation for any input data set, and provide a constant size witness for one or more members of the set, so that the membership of data can be verified.

Let $k \in \mathbb{N}$ be a security parameter, and p and q be two suitably-large strong primes where $p, q > 2^{\frac{3}{2}k}$, so $p \cdot q > 2^{3k}$. Let $N = p \cdot q$, $\varphi(N) = (p - 1) \times (q - 1)$, and α be a chosen base that is relatively prime to N .

Assuming that $S = \{x_1, \dots, x_n\}$ be a set with n element and $x_i \in \{0, 1\}^k$, for each element $x_i \in S$, there is an unique prime number $P(x_i)$ corresponding to it, where $P(*)$ is a prime generator. The accumulator of S is defined as $Acc(S) = \alpha^{\prod_{x_i \in S} P(x_i)} \bmod N$. For any subset $S' \subseteq S$, the proof is $\pi = \alpha^{\prod_{x_i \in S-S'} P(x_i)} \bmod N$. To verify whether $S' \subseteq S$ holds, we only need to check whether $Acc(S) = \pi^{\prod_{x_i \in S'} P(x_i)} \bmod N$ holds.

The security of RSA accumulator is guaranteed by strong RSA assumption which is defined as follows.

Strong RSA assumption [32]. Given $N = p \cdot q$ and choose a number $x \in \mathbb{Z}_N$, it is infeasible to find α and f such that $\alpha^f = x \bmod N$.

With strong RSA assumption, given N , α and $S = \{x_1, \dots, x_n\}$, it is infeasible to find $x \notin S$ and π such that $Acc(S) = \pi^{P(x)} \bmod N$.

In this paper, we use Euler's theorem and its corollary to complete some necessary work. Their definitions are as follows:

Euler's Theorem Given $\alpha > 1$ and $N > 1$, if they are relatively prime, then $\alpha^{\varphi(N)} \bmod N = 1$.

Corollary 1 Given $\alpha > 1$ and $N > 1$ are relatively prime, then $\alpha^x \bmod N = \alpha^{x \bmod \varphi(N)} \bmod N$, for all $x \geq 0$, where $\varphi(N) = (p - 1) \times (q - 1)$.

3.2 Patricia tree

Patricia tree is a compressed binary query tree, which is an index structure improved from Trie tree. The leaf nodes of Patricia tree record the corresponding information of keywords, and each path from the root node to a leaf node represents a keyword. Different from Trie

tree, the key of each node in Patricia tree records one or more elements of a keyword, which reduces the depth of the tree. In Ethereum [33], Patricia tree is modified and used for data verification. We adopt the modified Patricia tree as the index structure of the proposed scheme. Each keyword is described as a hexadecimal string, and it is stored in multiple nodes of a path from the root node to an internal node or a leaf node in the Patricia tree. There are three types of nodes in a modified Patricia tree, which is shown as follows:

- **Extension Node:** $(type, key, value)$. An extension node is a three-element tuple, where $type = \mathbf{EN}$, key stores the shared prefix of multiple keywords, and $value$ stores the pointer to its child node.
- **Branch Node:** $(type, hex[0,1,\dots,f], value)$. A branch node is a three-element tuple, where $type = \mathbf{BN}$ and $hex[0,1,\dots,f]$ represent the 16 hexadecimal characters from ‘0’ to ‘f’, respectively. Each item of hex stores a pointer to a child node. If the keyword represented by the path from the root node to the parent node of the branch node exists, then $value$ stores the corresponding value to the keyword, otherwise $value = \emptyset$.
- **Leaf Node:** $(type, key, value)$. An leaf node is also a three-element tuple, where $type = \mathbf{LN}$, key stores the last few elements that a keyword does not share with other keywords and $value$ stores the corresponding value to the keyword.

We give an example to illustrate the modified Patricia tree. Assuming that there are two keys {a111, a1f3} and the corresponding values {‘see’, ‘saw’}, the formed Patricia tree is shown in Figure 1.

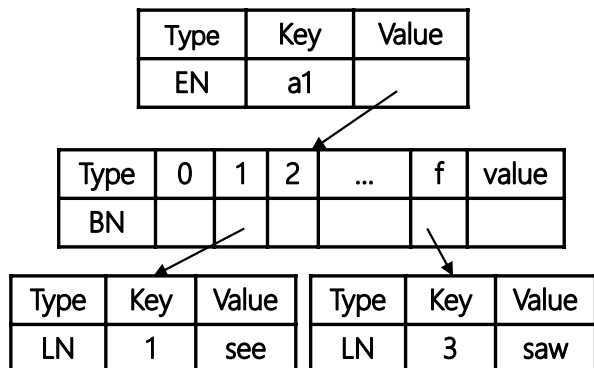
4 Models and problem statement

4.1 Notations

In this paper, given a string s , its length is expressed as $|s|$, and its first t elements are denoted as $prefix(s,t)$. Other notations used in this paper are shown as follows.

- D : The set of n documents, $D = \{d_1, d_2, \dots, d_n\}$.
- W : The keyword dictionary with a set of m keywords, $W = \{w_1, w_2, \dots, w_m\}$, which are extracted from the documents of D .

Figure 1. An example of Patricia tree



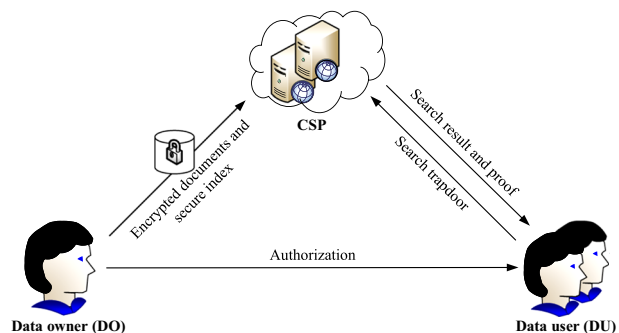
- $\bar{D}$: The set of n encrypted documents, $\bar{D} = \{\bar{d}_1, \bar{d}_2, \dots, \bar{d}_n\}$, where $\bar{d}_i$ is the ciphertext form of d_i .
- $id(*)$: The identity of a document. For example, $id(d_i)$ represents the identity of the document d_i .
- L_i : The posting list that stores the documents having the keyword w_i , $L_i = \{id(d_j) | d_j \in D \wedge w_i \in d_j\}$.
- ζ : The set of m posting list, $\zeta = \{L_1, L_2, \dots, L_m\}$, corresponding to the set of m keywords of W .
- PL : The auxiliary matrix, an $m \times n$ -dimensional matrix and used for generating proofs.
- α : The base of RSA accumulator.
- Ψ : The local evidence.
- π : The set of proof, $\pi = \{\pi_1, \pi_2, \dots, \pi_m\}$, where π_i corresponds to the keywords w_i .
- T_S : The search trapdoor of a query.
- C_R : The search result of a query.
- π_R : The proof of search result.
- $P(*)$: The prime number generator.
- $token(*)$: The one-way and collision-free token generator for keywords.

4.2 System model and threat model

We adopt the same system model as [29–31] in this paper, which is shown in Figure 2. There are three different entities: the data owner (DO), the data user (DU), and the cloud service provider (CSP). They collaborate as follows:

- 1) DO first constructs the posting list set ζ by processing the documents of D , then DO encrypts D into $\bar{D}$ and generates local evidence Ψ based on the keyword set W and the encrypted document set $\bar{D}$. After building a secure index I , DO submits $\bar{D}$ and I to CSP.
- 2) CSP stores the encrypted documents and index uploaded by DO. When CSP receives a search trapdoor from DU, it executes a search processing and returns the search result to DU.
- 3) DU generates a search trapdoor when a query is started, and it sends the trapdoor to CSP as a query command. When a query is finished and the search result is returned from CSP to DU, DU verifies the result by using the RSA accumulator verification. If

Figure 2. System model



the result passes the verification, it is accepted and decrypted by DU. Otherwise, it is dropped.

The threat model adopted in this paper is the same as that in [34]. DO and DU are assumed to be trusted, but CSP is not. CSP performs search processing on the encrypted data, but it could also try to obtain private information from ciphertext and trapdoors. On the other hand, CSP may suffer from external attacks which could result in data forgery, and CSP may return incomplete or incorrect search results for saving computing resources.

4.3 Design goals

Given a search request Q , the design goals of the EVSS scheme are demonstrated as follows.

Keyword search In the EVSS scheme, after the user submits a keyword search request, CSP can return the ciphertext set of the documents containing the keyword.

Search efficiency and accuracy The EVSS scheme using the RAP-tree index is designed to achieve efficient and accurate search.

Privacy preservation In our scheme, the privacy information of data is protected from being leaked to CSP. We aim to satisfy following privacy requirements:

- **Confidentiality:** Only DO and DU can obtain the private information of documents and keywords, and CSP cannot infer any plaintext information of them.
- **Verifiability:** If CSP returns the forged or incomplete search results, DU can detect the dishonesty of CSP through the result verification.

4.4 Scheme definition

To introduce our scheme concisely, we show the definition of each algorithm in the scheme in this section. The EVSS scheme consists of two modules, setup module search module. The first module is to encrypt documents and build index, which includes algorithms *GenKey*, *Encrypt*, *GenProof* and *BuildIndex*. And the second module is to perform the search and verify the search results, which includes algorithms *GenSearchTD*, *Search* and *Verify*. The definitions of each algorithm are as follows:

- *GenKey*: DO generates the secret key SK and the public parameters $params$.
- *Encrypt*: DO encrypts the documents and generates a posting list set ζ .
- *GenProof*: DO generates the auxiliary matrix PL and calculates the local evidence Ψ . Then, DO calculates the proof set π .
- *BuildIndex*: DO generates a key-value pair for each keyword and inserts it into the RAP-tree to build the index I .
- *GenSearchTD*: DU generates the search trapdoor T_S .
- *Search*: CSP performs search and returns the search results and proof.
- *Verify*: DU verifies the search results. If the verification is successful, it accepts and decrypts the ciphertext set. Otherwise, it drops the search results.

5 The verifiable search scheme

In this section, we give the details of the Efficient Verifiable Search Scheme (EVSS). It is consisting of two modules: the setup module and the search module. In the setup module, DO implements the calculation of local evidence and the construction of index. DO first extracts effective keywords from the document set and generate the posting list set. Then it calculates the local evidence and the proof of each keyword. After that DO builds an RSA Accumulator based Patricia tree index and sends it to CSP. In the search module, CSP implements the search of keyword and DU verifies the search results. We introduce our EVSS scheme module by module.

5.1 Setup module

In this module, data are pre-processed to setup the verifiable search system, where four algorithms *GenKey*, *Encrypt*, *GenProof* and *BuildIndex* introduced as follows.

$\{SK, params\} \leftarrow \mathbf{GenKey}(1^k)$: DO chooses two suitably-large prime $p, q > 2^{\frac{3}{2}k}$ and a base α for RSA accumulator so that it can get $N = p \cdot q$ and $\varphi(N) = (p - 1) \times (q - 1)$. Then, DO generates a private key k_s for symmetric encryption. Finally, DO sets the parameters $params = \{N, \alpha\}$ and the private keys $SK = \{p, q, k_s\}$.

$\{\bar{D}, \zeta\} \leftarrow \mathbf{Encrypt}(SK, D)$: The *Encrypt* algorithm is run by DO to generate the encrypted document set $\bar{D}$ and the posting list set ζ . In the algorithm, DO first extracts keywords from documents. Then, DO generates a posting list L_i for each keyword $w_i \in W$, where the identity of a document is added into L_i if the document contains w_i . At the same time, DO uses a symmetric encryption algorithm to encrypt documents. The pseudo-code of *Encrypt* is shown in Algorithm 1.

Require: the set of private keys SK , and the set of input documents D .

Ensure: the set of encrypted documents $\bar{D}$, and the posting list set ζ .

```

1: Initialize  $\bar{D} = \emptyset$  and  $\zeta = \emptyset$ ;
2: Extract keywords from documents of  $D$  and generate the keyword set  $W$ ;
3: for each  $w_i \in W$  do
4:   Create a empty posting list  $L_i = \emptyset$  corresponding to  $w_i$ ;
5: end for
6: for each  $d_i \in D$  do
7:   Encrypt  $d_i$  into  $\bar{d}_i$  by using the key  $k_s$  of  $SK$ , and then add  $\bar{d}_i$  into  $\bar{D}$ ;
8:   for each  $w_j \in d_i$  do
9:     Append  $id(d_i)$  to the end of  $L_j$ ;
10:  end for
11:  Add  $L_j$  into  $\zeta$ ;
12: end for
13: return  $\bar{D}$  and  $\zeta$ ;

```

Algorithm 1 *Encrypt* (SK, D).

$\{PL, \Psi, \pi\} \leftarrow \text{GenProof}(\overline{D}, \zeta, \alpha)$: The *GenProof* algorithm is run by DO to generate the auxiliary matrix PL , the local evidence Ψ and the proof set π . In the algorithm, DO first initializes the local evidence $\Psi = \alpha$ and then creates an empty $m \times n$ -dimensional matrix. Afterwards, each posting list of ζ is traversed, and PL is assigned values according to (1),

$$PL[i][j] = \begin{cases} P(H(w_i, H(\overline{d_j}))), & w_i \in d_j \\ \emptyset, & \text{otherwise} \end{cases} \quad (1)$$

where $P(*)$ is a unique prime generator and $H(*)$ is a collision-free hash function. The unique prime number stored in $PL[i][j]$ represents that the keyword w_i exists in the encrypted document $\overline{d_j}$. The local evidence Ψ is calculated by adding effective elements in PL to RSA accumulator, and the calculation is shown in (2),

$$\Psi = \alpha^{\prod_{w_i \in W} \prod_{d_j \in D(w_i)} PL[i][j]} \bmod N, \quad (2)$$

where α is the base of RSA accumulator and $D(w_i)$ denotes the set of documents containing the keyword w_i . The proof $\pi = \{\pi_1, \pi_2, \dots, \pi_m\}$ is calculated by using (3),

$$\pi_i = \Psi^{\frac{1}{\prod_{d_j \in D(w_i)} PL[i][j]} \bmod \varphi(N)} \bmod N. \quad (3)$$

The pseudo-code of *GenProof* is shown in Algorithm 2.

Require: the set of encrypted documents $\overline{D}$, the posting list set ζ , and the base of RSA accumulator α .

Ensure: the local evidence Ψ , the auxiliary matrix PL , and the proof set $\pi = \{\pi_1, \pi_2, \dots, \pi_m\}$.

```

1: Initialize  $\Psi = \alpha$  and create an empty  $m \times n$ -dimensional matrix  $PL$ ;
2: for each  $L_i \in \zeta$  do
3:    $\pi_i = 1$ ;
4:   for each  $id(d_j) \in L_i$  do
5:      $PL[i][j] = P(H(w_i, H(\overline{d_j})))$ ;
6:      $\Psi = \Psi^{PL[i][j]} \bmod N$ ;
7:   end for
8: end for
9: for each  $w_i \in W$  do
10:   $\pi_i = \Psi^{\frac{1}{\prod_{d_j \in D(w_i)} PL[i][j]} \bmod \varphi(N)} \bmod N$ ;
11: end for
12: return  $PL$ ,  $\Psi$  and  $\pi$ ;

```

Algorithm 2 GenProof ($\overline{D}, \zeta, \alpha$).

$I \leftarrow \text{BuildIndex}(W, \pi, \zeta)$: The *BuildIndex* algorithm is executed by DO to construct the RSA accumulator-based Patricia tree (RAP-tree) index. In this algorithm, for each keyword w_i , DO first generates the corresponding token $token(w_i)$ as the key variable K and then it constructs the proof π_i and the posting list L_i from π and ζ , respectively. Afterwards, the value variable V is assigned $\{\pi_i, L_i\}$. Finally, the key-value pair (K, V) corresponding to w_i is added into the RAP-tree index I . The pseudo-code of *BuildIndex* is shown in Algorithm 3.

Require: the keyword dictionary W , the proof set π , and the posting list set ζ .

Ensure: the generated RAP-tree index I .

- 1: Initialize the RAP-tree index $I = \emptyset$ and the variables $K = \emptyset, V = \emptyset$;
- 2: **for** each keyword $w_i \in W$ **do**
- 3: $K = \text{token}(w_i)$;
- 4: Get the corresponding proof π_i from π and the corresponding posting list L_i from ζ according to the keyword w_i ;
- 5: $V = \{\pi_i, L_i\}$;
- 6: Add the pair (K, V) into I according to the Patricia tree construction algorithm in [33];
- 7: **end for**
- 8: **return** I ;

Algorithm 3 BuildIndex(W, π, ζ).

After the above algorithms in the setup module have been accomplished, DO outsources the encrypted documents $\bar{D}$ and the RAP-tree index I to CSP and shares SK , $papams$ and Ψ with DU.

5.2 Search Module

This module is to perform verifiable searches by the interactions between DU and CSP, where three algorithms *GenSearchTD*, *Search* and *Verify* are presented.

$T_S \leftarrow \text{GenSearchTD}(w_i, SK)$: The *GenSearchTD* algorithm is executed by DU when it starts a keyword search. Assuming that DU wants to search the documents that contain the keyword w_i , DU uses the one-way and collision-free token generator to generate the search trapdoor $T_S = \text{token}(w_i)$ and then it transmits T_S as the search command to CSP.

$\{C_R, \pi_R\} \leftarrow \text{Search}(I, T_S)$: After receiving the search trapdoor $T_S = \text{token}(w_i)$ from DU, CSP executes the search algorithm by using the RAP-tree index I . We use the notation *pstring* to denote the hexadecimal character string of a path from the root node to an internal node or a leaf node in I . In the depth-first search processing over I , when *pstring* is consistent with the token in T_S , if the end node of the search path is a leaf node or a branch node whose *value* is not null, then the search is successful. CSP gets the posting list L_R and the proof π_R from the end node of the search path, and then it return C_R and π_R to DU as the search result, where C_R is the set of encrypted documents whose identities are in L_R .

$\{\text{true/false}\} \leftarrow \text{Verify}(C_R, \pi_R, \Psi)$: The *Verify* algorithm is run by DU to verify the completeness of the search result. When DU receives the search result $\{C_R, \pi_R\}$ from CSP for the search trapdoor $T_S = \text{token}(w_i)$, it calculates the verification code Ψ^* according to (4),

$$\Psi^* = (\pi_R)^{\prod_{c_j \in C_R} P(H(w_i, H(c_j)))} \bmod N. \quad (4)$$

If $\Psi^* = \Psi$ where Ψ is the shared local evidence between DU and DO, the result verification passes and the documents in C_R are decrypted to generate the plaintext search result;

otherwise, the result verification fails and the search result is dropped. Detailed procedures of *Verify* is shown in Algorithm 4.

Require: Ψ is the evidence, C_R is a collection of ciphertexts returned by CSP, and π_R is the proof for verifying search results.

Ensure: the verification result *true/false*.

- 1: Compute $\Psi^* = (\pi_R)^{\prod_{\bar{d}_j \in C_R} P(H(w_i, H(\bar{d}_j)))} \bmod N$;
- 2: **if** $\Psi^* = \Psi$ **then**
- 3: Accept the search result and decrypt the ciphertext;
- 4: **return** *true*;
- 5: **else**
- 6: Drop the search result;
- 7: **return** *false*;
- 8: **end if**

Algorithm 4 $\text{Verify}(C_R, \pi_R, \Psi)$.

5.3 Illustration example

We give an example to illustrate the algorithms of the EVSS scheme, which is shown in Figure 3. We assume that there are three documents $D = \{d_1, d_2, d_3\}$ with four keywords $W = \{w_1, w_2, w_3, w_4\}$ which is contained in $\{d_1, d_3\}$, $\{d_3\}$, $\{d_2\}$ and $\{d_1, d_2, d_3\}$, respectively. The corresponding tokens of four keywords are $\{b59, b59732, b59a5, 7a48\}$, respectively.

(1) When the search system is setup, DO performs the initial operation before data outsourcing. According to the posting list set, DO can construct the auxiliary matrix PL , and the initial value of the elements in the matrix can be set to null. Then, DO encrypts the document set into ciphertext set $\{c_1, c_2, c_3\}$ with the given key SK . For each keyword-ciphertext

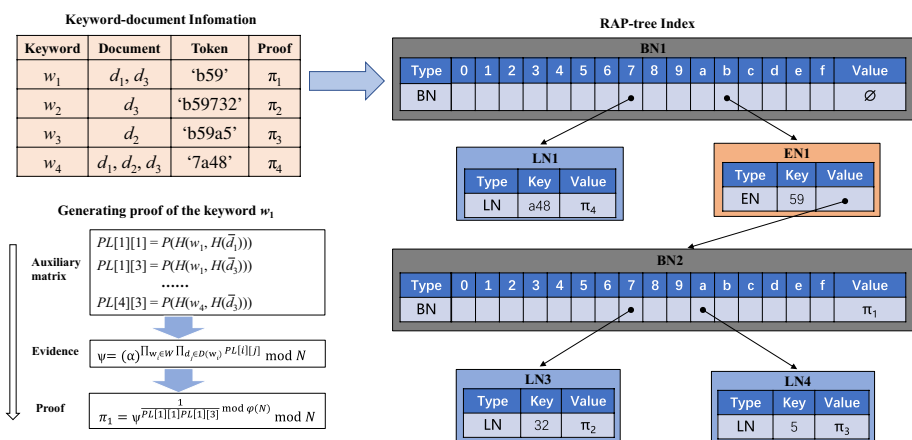


Figure 3. Illustration Example

pair $\{w_i, c_j\}$, DO computes the corresponding element $PL[i][j] = P(H(w_i, H(\overline{d_j})))$ in the auxiliary matrix PL . Then, DO can compute the local evidence Ψ as:

$$\Psi = (\alpha)^{\prod_{w_i \in W} \prod_{d_j \in D(w_i)} PL[i][j]} \bmod N.$$

For each keyword, DO calculates its corresponding proof, taking keyword w_1 as an example:

$$\pi_1 = \Psi^{\frac{1}{PL[1][1] \cdot PL[1][3]} \bmod \varphi(N)} \bmod N.$$

DO takes the hexadecimal token converted by the keyword as the *key*, takes the identifies of all documents containing the keyword and corresponding proof π_i as the *value*, and inserts the key-value pair into the index. After inserting all the key-value pairs, DO gets the index I . Finally, it sends the index I and the encrypted document set C to the CSP.

(2) When a query is started, CSP performs the search procedure and returns the search results, and DU verifies the integrity of the search results. Assuming that DU wants to search a keyword w_1 that already exists in the index, it sends the trapdoor corresponding to w_1 to the CSP. After receiving the trapdoor, CSP executes the search algorithm, and the search path is $\{BN1, EN1, BN2\}$. CSP gets the proof π_1 and the ID of the documents $\{d_1, d_3\}$ containing w_1 from the terminal node of the path. Subsequently, the CSP sends the retrieved document ciphertext set and π_1 to the DU. Once DU receives the ciphertext set and proof, it computes $x_1 = P(H(w_1, H(c_1)))$ and $x_2 = P(H(w_1, H(c_3)))$, then it reconstructs accumulator as

$$\Psi^* = \pi_1^{(x_1 \cdot x_2)} \bmod N,$$

if $\Psi^* = \Psi$ holds, the search results are correct and DU can decrypt $\{c_1, c_3\}$ into $\{d_1, d_3\}$, otherwise, it drops the search results.

6 Security analysis

In this section, we give the security analysis of the EVSS scheme. We analyze the confidentiality and verifiability of the scheme. Confidentiality means that the adversary cannot obtain the privacy information of documents and keywords through index, while verifiability means that the retrieval results received from the CSP can be verified and be accepted only when the verification passes. We adopt the Real/Ideal simulation [35] to prove the confidentiality of EVSS.

Definition 1 (EVSS confidentiality) Let the EVSS scheme be a verifiable scheme based on searchable symmetric encryption and consider the following probabilistic experiments, where $\mathcal{A}$ is an adversary, $\mathcal{S}$ is a simulator, and $\mathcal{L}$ are leakage algorithms:

Real_A(k): A challenger runs $GenKey(1^k)$ to generate symmetric key k_s . The adversary $\mathcal{A}$ selects a document set for the challenger, who runs the algorithm $BuildIndex(\zeta, \pi)$ to generate indexes I . With distinct keywords as input, the adversary makes a polynomial number of queries, and for each, $\mathcal{A}$ can get a search trapdoor T_s from the Challenger through the algorithm $GenSearchTD(w, SK)$. Finally, $\mathcal{A}$ returns a bit.

Ideal_{A,S}(k): The adversary $\mathcal{A}$ selects a document set D . Given $\mathcal{L}(D)$, the simulator $\mathcal{S}$ generates and sends index I , ciphertext set C to $\mathcal{A}$. $\mathcal{A}$ makes a polynomial number of queries. For each query, $\mathcal{S}$ returns the appropriate trapdoor T_S . Finally, $\mathcal{A}$ returns a bit in this experiment.

We say that the EVSS scheme satisfies confidentiality if for any probabilistic polynomial-time (PPT) adversary $\mathcal{A}$, there exists a PPT simulator $\mathcal{S}$ such that

$$|Pr[Real_{\mathcal{A}} = 1] - Pr[Ideal_{\mathcal{A},S} = 1]| \leq \text{negl}(k). \quad (5)$$

Theorem 1 The EVSS scheme satisfies confidentiality in Definition 1.

Proof We show that exist a polynomial-time simulator $\mathcal{S}$ such that for any probabilistic polynomial-time (PPT) adversary $\mathcal{A}$, the output between the real game and a simulation game is computationally indistinguishable.

Given the leakage function $\mathcal{L}_1(\mathcal{D}) = (|C|, |I|, \{|d|\})$, $\mathcal{L}_2(\mathcal{D}) = (\{T_S\}, \{path\})$, where $|C|$ is the size of ciphertext set, $|I|$ is the size of the index indicated by the number of terminal nodes, $\{|d|\}$ is the size of each document in the document set D , $\{T_S\}$ are q generated search tokens, and $\{path\}$ are the search paths.

$\mathcal{S}$ first simulates the ciphertext $\tilde{c}_j = Enc(sk, |d_j|)$, where sk is randomly selected for the CPA-secure encryption algorithm Enc , and $|d_j|$ is revealed by $\mathcal{L}(\mathcal{D})$. Meanwhile, $\mathcal{S}$ simulates the index $\tilde{I}$ by choosing $|I|$ random key-value strings and inserting into the tree. (C, I) and $(\tilde{C}, \tilde{I})$ are indistinguishable because Enc is CPA-secure and each key-value pair in the index is generated by pseudo-random function.

Next, $\mathcal{S}$ simulates keyword searches. The adversary $\mathcal{A}$ makes a polynomial number of queries by picking keywords. In the case that the keyword w has not been searched. If the token of w matches a path in I , $\mathcal{S}$ will randomly select a path in $\tilde{I}$ and return it to $\mathcal{A}$ as $\tilde{T}_w$. Otherwise, $\mathcal{S}$ randomly returns a string that does not match any path in $\tilde{I}$ to $\mathcal{A}$. Therefore, $\mathcal{A}$ cannot distinguish between T_w and $\tilde{T}_w$ by their search process. Moreover, since the search trapdoors are generated by a pseudo-random function, we can say that $\mathcal{A}$ can't distinguish T_w and $\tilde{T}_w$. If the keyword has been searched, $\mathcal{S}$ only needs to return the same token as before. Since adversary $\mathcal{A}$ can't distinguish between real trapdoor and simulated trapdoor, it can be concluded that real outputs and simulated outputs are also indistinguishable for $\mathcal{A}$. $\square$

The verifiability of EVSS means that our scheme can verify the integrity and the availability of the search results, so as to prevent the data integrity attack. Here, we prove the verifiability of EVSS.

Definition 2 (EVSS verifiability) Let the EVSS scheme be a verifiable scheme based on searchable symmetric encryption and consider the following probabilistic experiments, where $\mathcal{A}$ is a adversary:

Vrf_A(k):

1. the challenger runs $GenKey(1^k)$ to generate the symmetric key k_s .
2. the adversary $\mathcal{A}$ selects a document set D for the challenger.
3. the challenger runs $BuildIndex(\zeta, \pi)$ to generate the verifiable index I , and runs $GenEvidence(C, \zeta)$ to generate the evidence Ψ .
4. given the index I , the evidence Ψ and access to $GenSearchTD(w, SK)$, the adversary $\mathcal{A}$ outputs a search trapdoor T_S , a collection of ciphertexts C' where $C' \neq C_w$, and a proof π' , where C_w is the encrypted documents that contain the keyword w .

5. the challenger runs $\text{Boolean} = \text{Verify}(\pi, \Psi)$.
6. the output of the experiment is the Boolean.

We say that the EVSS scheme is verifiable if for all PPT adversaries $\mathcal{A}$,

$$\Pr[\text{Verf}_{\mathcal{A}} = 1] \leq \text{negl}(k) \quad (6)$$

Theorem 2 The EVSS scheme is verifiable if the hash function H is collision-resistant and P is pseudo-random.

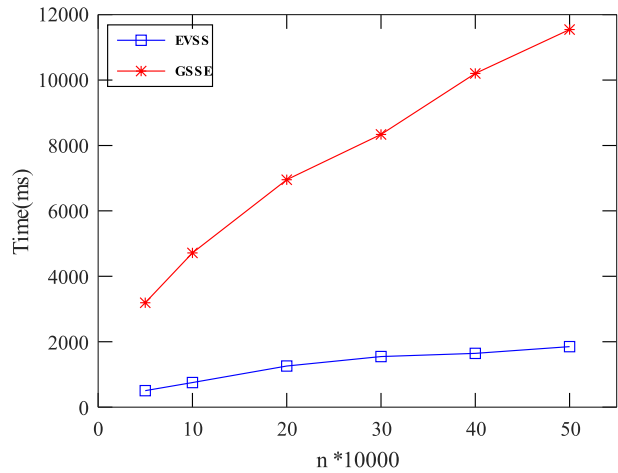
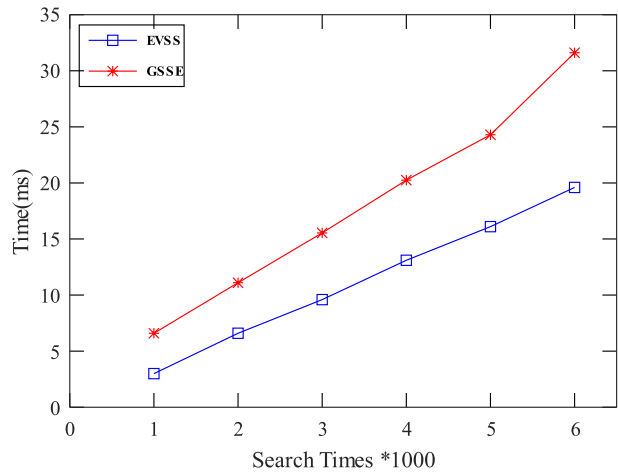
Proof Assuming that $\mathcal{A}$ forges a ciphertext set $\tilde{C}_w$ as the search result of keyword w as well as a proof $\tilde{\pi}_w$ such that Verify returns *true* when $(\tilde{C}_w, \tilde{\pi}_w)$ is input. Assuming that the correct search result is C_w , and the proof is π_w . We will prove that $\mathcal{A}$ cannot forge $(\tilde{C}_w, \tilde{\pi}_w) \neq (C_w, \pi_w)$ that makes Verify return *true*. Firstly, the evidence Ψ is constructed and preserved by the data owner, and sent to DU after being encrypted. Any entity without secret key cannot forge the same evidence, and the digital signature ensures that DU only accepts the authenticated evidence. Secondly, $(\tilde{C}_w, \tilde{\pi}_w) \neq (C_w, \pi_w)$ means we need to consider three different cases: $\tilde{C}_w \neq C_w$ and $\tilde{\pi}_w = \pi_w$; $\tilde{C}_w = C_w$ and $\tilde{\pi}_w \neq \pi_w$; $\tilde{C}_w \neq C_w$ and $\tilde{\pi}_w \neq \pi_w$. In the process of accumulator reconstruction, the prime number generated for each keyword-ciphertext pair is guaranteed to be unique by H and P . π is stored in encrypted form in the server. Therefore, based on the strong RSA assumption, we can get that the accumulator reconstructed with $(\tilde{C}_w, \tilde{\pi}_w)$ cannot be the same as the evidence Ψ sent by DO whether in the case of $\tilde{C}_w \neq C_w$ or $\tilde{\pi}_w \neq \pi_w$. So the verifiability of the Verify algorithm is guaranteed. $\square$

According to Theorem 1, Theorem 2 and their proofs, we can conclude that the EVSS scheme guarantees the confidentiality of documents and keywords, and at the same time realizes the verification of the integrity of search results. The keyword is encrypted in the process of generating the token in hexadecimal format, so CSP cannot infer any private information of the keyword through its token. CSP also cannot obtain the plaintext information of the data through the leaking function. The EVSS scheme can detect manipulation, forgery, malicious deletion and other active attacks.

7 Performance evaluation

In the experiment, we implement the proposed scheme EVSS and the compared scheme GSSE [29]. GSSE is a representative scheme in the field of verifiable keyword search over encrypted documents, which adopts the MPT tree to achieve verifiable keyword search. Performance on time cost and space cost of two schemes are evaluated. We conducted experiments on the same data set at the same local PC. Each experiment is performed 10 times and the average is taken as the result.

Experiments were conducted on a local machine running the Microsoft Windows 10 Ultimate operating system with an Inter Core i5 CPU running at 4.3GHz and 16GB memory. The programs were implemented in Java and compiled using IntelliJ IDEA. The cryptographic algorithms were implemented with JPBC library. To validate the effectiveness and efficiency of the schemes in practice, we use the New York Times Dataset[36] as the test dataset. This data set has about 500,000 news with a total size of about 1.81GB. The average size of each document is about 3.8KB. We extract

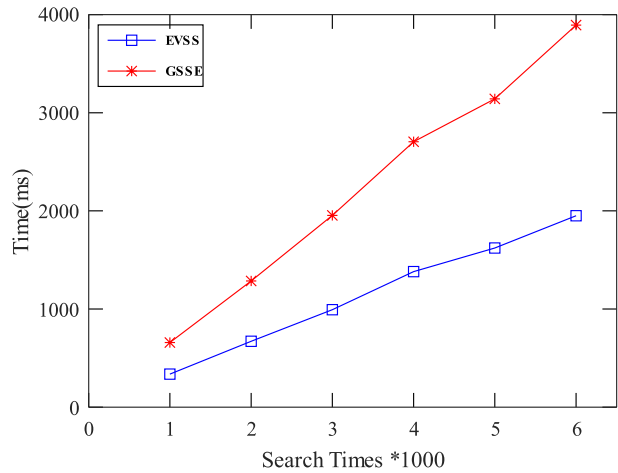
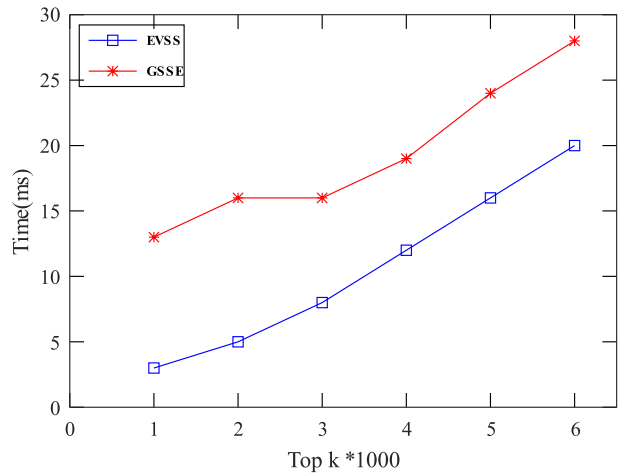
Figure 4. Index Construction Time**Figure 5.** Search Time

meaningful keywords from each document and get 108678 keywords as a dictionary. Detailed experiment result is shown in the following two subsections.

7.1 Time cost evaluation

We perform the time cost evaluation from five aspects, the index construction time cost, the search time cost, the search result verification time cost, the top- k search time cost and the top- k search result verification time cost. The experiment results are shown in Figures 4, 5, 6, 7 and 8.

Figure 4 shows the time cost of building the index. The time cost of GSSE index construction is significantly higher than that of EVSS, and the gap becomes larger as the number of documents increases. The reason is that the time cost of building an index in both schemes are positively related to the number of documents. When a document is processed

Figure 6. Verification Time**Figure 7.** Top k Search Time

and added into the tree indexes, GSSE needs to update the hash values of ancestor nodes of the node having the document, while EVSS does not need to perform these operations.

Figure 5 shows the time cost of keyword search in two schemes. The search time by using the tree indexes are both positively related to the length of the search path from the root to the node corresponding to the queried keyword. Since the time cost of a single search is too short to measure accurately, we randomly choose the queried keyword multiple times to perform multiple searches, and we record the total search time cost. In Figure 5, GSSE has a higher search time cost than EVSS. The reason is that GSSE requires more data read operations. In the search processing, EVSS compares the search token with the key of the node in the index. If any node matches, EVSS goes to the next layer to continue the search until it searches to the terminal node or does not match any key of node. After the search is completed, CSP sends the search results to DU. In the process of GSSE performing the search operation, it needs to generate a proof for the search results, that is, the hash values needed to reconstruct the root node hash.

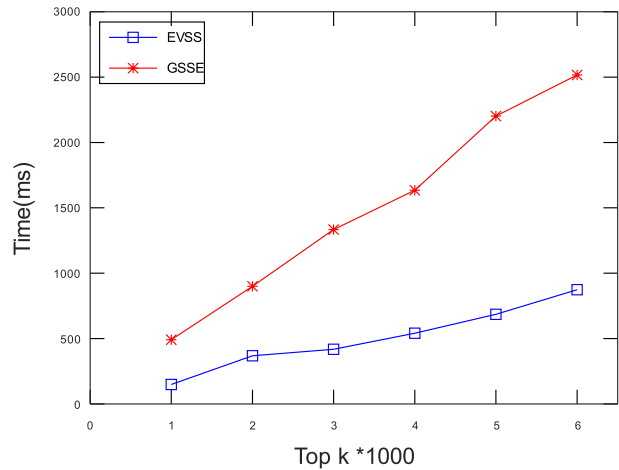
Figure 8. Top k Verification Time

Figure 6 shows the time cost of DU verifying the search results. The time cost of GSSE to verify search results is higher than that of EVSS, and with the increase of search times, EVSS saves more time. The reason is that EVSS reconstructs the RSA accumulator by modular exponentiation, and GSSE reconstructs the root node hash by incremental hash calculation, but GSSE performs more incremental hashes. It takes less time to reconstruct the RSA accumulator when the searched keywords are the same.

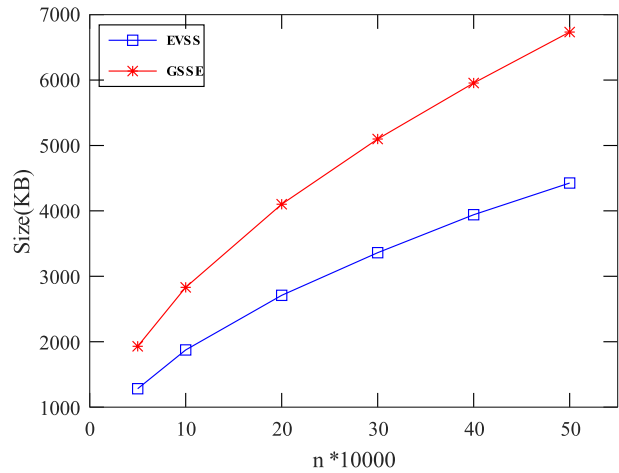
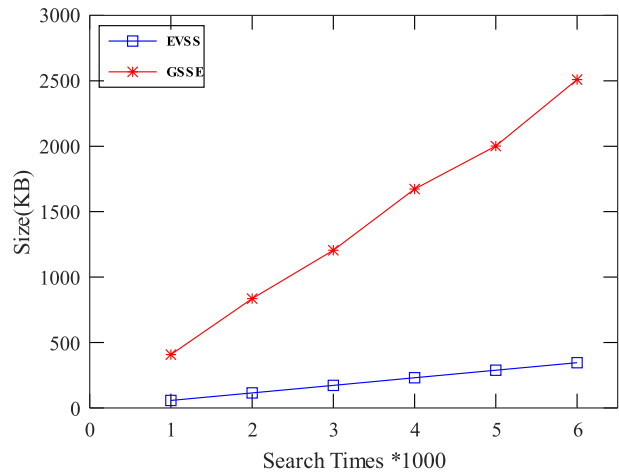
In order to test the performance of the two schemes in the retrieval process of the keywords with the highest frequency in the document set, we rank the keywords according to the frequency in the document set, and select the keywords with the highest ranking for experiments. Figures 7 and 8 respectively show the performance of the two schemes when searching and verifying the top- k keywords with the highest ranking. According to the analysis above and the display of figure, we find that the EVSS scheme has better performance when searching and verifying for keywords with high frequency.

7.2 Space cost evaluation

We evaluate the space cost of the index and proof in the two schemes. The experiment results are shown in Figs 9, 10 and 11.

Figure 9 shows the storage cost of the index. It shows that the storage cost of EVSS indexes is relatively lower than that of GSSE. The reason is that the storage cost of both schemes are positively related to the number of nodes, but GSSE requires additional space to store the hash value of the nodes.

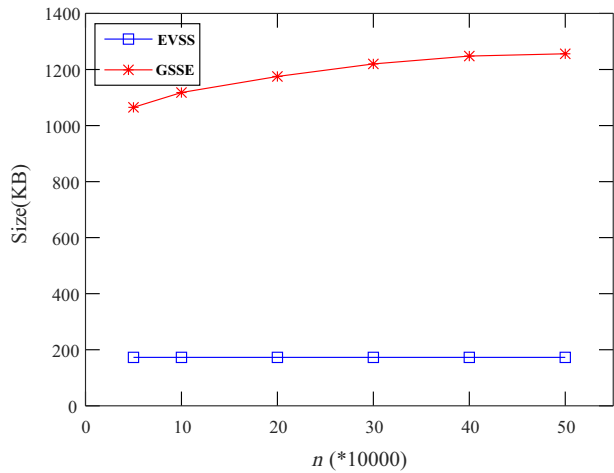
Figures 10 and 11 show the space cost of proofs returned by CSP. Figure 10 shows that as the number of searches increases, the space cost of proofs for both EVSS and GSSE increases linearly, but EVSS is significantly lower than GSSE. Figure 11 shows that as the number of documents n increases, the space cost of EVSS proof does not

Figure 9. Index Size**Figure 10.** Proof Size

change, but the GSSE increases. The reason is that the proof of each search in EVSS is a fixed-length encrypted large number, which is independent of the size of the database, while the proof in GSSE is composed of the hash values of the neighbor elements of the elements on the search path. Obviously, the space cost of the proof information of EVSS is smaller, and the communication cost between the DU and CSP consumed by the EVSS is lower.

8 Conclusion

In this paper, we design an EVSS scheme which supports efficient verifiable keyword search on ciphertext. The experimental results demonstrate that our scheme has efficient search rate, and the RSA accumulator can effectively detect the malicious modification and

Figure 11. Proof Size

destruction of the data. At the same time, the communication consumption between the cloud server and DU for verifying the search results is small. As our future work, we plan to design a more concise and convenient VSSE scheme, which can effectively verify the ranking search on ciphertext.

Author contributions This work thanks the following authors for their contributions: Qian Zhou and Hua Dai contributed to the conception of the study; Qian Zhou and Wenjie Sheng performed the experiment; Qian Zhou and Yuanlong Liu contributed significantly to analysis and manuscript preparation; Hua Dai and Wenjie Sheng performed the data analyses and wrote the manuscript; Geng Yang helped perform the analysis with constructive discussions.

Funding This work is supported by the National Natural Science Foundation of China under the grant Nos.61872197, 61972209, 61902199 and 61771251; the Postdoctoral Science Foundation of China under the Grand No. 2019M651919; the Postgraduate Research and Practice Innovation Program of Jiangsu Province under the Grand No. KYCX22_0984.

Data availability Details of the datasets have been described in Section 7.

Declarations

Ethics approval and consent to participate Our manuscripts were not submitted to multiple journals for simultaneous consideration and original. All authors agree with the content of the article.

Consent for publication Our manuscript is approved by all authors for publication.

Competing interests No conflict of interest exists in the submission of this manuscript, and manuscript is approved by all authors for publication.

References

1. Jiayun, H., Yu, L., He, C., Xiuxia, T., Cheqing, J.: An enhanced wildcard-based fuzzy searching scheme in encrypted databases. *World Wide Web* **23**(3), 2185–2214 (2020)

2. Qiang, C., Yanping, L., Zhenqiang, W., Yinbin, M., Jianqing, L.: Privacy-preserving conjunctive keyword search on encrypted data with enhanced fine-grained access control. *World Wide Web* **23**(2), 959–989 (2020)
3. Song, D.X., Wagner, D., Perrig, A.: Practical techniques for searches on encrypted data. In: *Proceedings of the 2000 IEEE Symposium on Security and Privacy*, pp 44–55. IEEE Computer Society (2000)
4. Goh, E.-J.: Secure indexes, IACR Cryptology ePrint Archive. **2003**, 1–18 (2004)
5. Curtmola, R., Garay, J., Kamara, S., Ostrovsky, R.: Searchable symmetric encryption: Improved definitions and efficient constructions. *J. Comput. Secur.* **19**(5), 895–934 (2011)
6. Tiwari, M.: Fuzzy keyword search over encrypted data in cloud computing. *International Journal of Research in Engineering & Technology* **4**(6), 15–20 (2016)
7. Sun, W., Shucheng, Y., Lou, W., Thomas, Y., Li, H.: Protecting your right: Verifiable attribute-based keyword search with fine-grained owner-enforced search authorization in the cloud. *IEEE Transaction on Parallel Distributed System* **27**(4), 1187–1198 (2016)
8. Zhang, W., Lin, Y., Xiao, S., Liu, Q., Zhou, T.: Secure distributed keyword search in multiple clouds. *IEEE, IWQoS* (2014)
9. Kurosawa, K., Ohtaki, Y.: Uc-secure searchable symmetric encryption. In: *Financial Cryptography and Data Security*, pp 285–298. Springer, Berlin (2012)
10. Pang, H., Jain, A., Ramamritham, K., Tan, K.-L.: Verifying completeness of relational query results in data publishing. In: *Proceedings of the ACM SIGMOD International Conference on Management of Data*, pp 407–418. Association for Computing Machinery (2005)
11. Narasimha, M., Tsudik, G.: Dsac: integrity for outsourced databases with signature aggregation and chaining. In: *Proceedings of the 2005 ACM CIKM International Conference on Information and Knowledge Management*, pp 235–236. Association for Computing Machinery (2005)
12. Chai, Q., Gong, G.: Verifiable symmetric searchable encryption for semi-honest-but-curious cloud servers. *IEEE* (2012)
13. Kumar, M., Pasupuleti, S.K.: Lightweight verifiable auditing for outsourced database in cloud computing. In: *Distributed Computing and Internet Technology - 14th International Conference*, pp 280–287. Springer (2018)
14. Yang, Y., Papadopoulos, S., Papadias, D., Kollios, G.: Authenticated indexing for outsourced spatial databases. *VLDB J.* **18**(3), 631–648 (2009)
15. Camenisch, J., Lysyanskaya, A.: Dynamic accumulators and application to efficient revocation of anonymous credentials. In: *Advances in Cryptology-CRYPTO 2002*, pp 61–76. Springer, Berlin (2002)
16. Chang, Y.-C., Mitzenmacher, M.: Privacy preserving keyword searches on remote encrypted data. In: *Applied Cryptography and Network Security*, vol. 3531, pp 442–455. Springer, Berlin (2005)
17. Boneh, D., Di Crescenzo, G., Ostrovsky, R., Persiano, G.: Public key encryption with keyword search. In: *Advances in Cryptology-EUROCRYPT 2004*, vol. 3027, pp 506–522. Springer, Berlin (2004)
18. Golle, P., Staddon, J., Waters, B.: Secure conjunctive keyword search over encrypted data. In: *Applied Cryptography and Network Security*, pp 31–45. Springer, Berlin (2004)
19. Boneh, D., Waters, B.: Conjunctive, subset, and range queries on encrypted data. In: *Theory of Cryptography*, pp 535–554. Springer, Berlin (2007)
20. Hore, B., Chang, E.-C., Diallo, M., Mehrotra, S.: Indexing encrypted documents for supporting efficient keyword search. In: *Secure Data Management-9th VLDB Workshop*, pp 93–110. Springer (2012)
21. Chuah, M., Hu, W.: Privacy-aware bedtree based solution for fuzzy multi-keyword search over encrypted data. In: *Proceedings of the 2011 31st International Conference on Distributed Computing Systems Workshops*, pp 273–281. IEEE Computer Society (2011)
22. Xu, P., Jin, H.: Public-key encryption with fuzzy keyword search: a provably secure scheme under keyword guessing attack. *IEEE Transaction on Computers* **62**(11), 2266–2277 (2013)
23. Wang, C., Ren, K., Yu, S.: Achieving usable and privacy-assured similarity search over outsourced cloud data. In: *Proceedings of the IEEE INFOCOM 2012*, pp 451–459. IEEE (2012)
24. Pang, H., Mouratidis, K.: Authenticating the query results of text search engines. *The Proceedings of the VLDB Endowment (PVLDB)* **1**(1), 126–137 (2008)
25. Zhang, Z., Wang, J., Yu, S., Chen, X.: Towards efficient verifiable forward secure searchable symmetric encryption. In: *Computer Security-ESORICS 2019*, vol. 11736, pp 304–321. Springer International Publishing (2019)
26. Gao, X., Yu, J., Chang, Y., Wang, H., Fan, J.: Checking only when it is necessary: Enabling integrity auditing based on the keyword with sensitive information privacy for encrypted cloud data. *IEEE Transactions on Dependable and Secure Computing* **PP** 1–17 (2021)

27. Hu, H., Xu, J., Chen, Q., Yang, Z.: Authenticating location-based services without compromising location privacy. In: Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data, pp 301–312. Association for Computing Machinery (2012)
28. Deepa, N., Perumal, P.: Hybrid context aware recommendation system for e-health care by merkle hash tree from cloud using evolutionary algorithm. *Soft. Comput.* **24**(10), 7149–7161 (2020)
29. Zhu, J., Li, Q., Wang, C., Yuan, X., Wang, Q., Ren, K.: Enabling generic, verifiable, and secure data search in cloud services. *IEEE Trans. Parallel Distrib. Syst.* **29**(8), 1721–1735 (2018)
30. Liu, Q., Nie, X., Liu, X., Peng, T., Wu, J.: Verifiable ranked search over dynamic encrypted data in cloud computing. In: 25th IEEE/ACM International Symposium on Quality of Service, pp 1–6. IEEE (2017)
31. Liu, Q., Tian, Y., Wu, J., Peng, T., Wang, G.: Enabling verifiable and dynamic ranked search over outsourced data. *IEEE Trans. Serv. Comput.* **15**(1), 69–82 (2022)
32. Goodrich, M., Tamassia, R., Hasic Telalovic, J.: An efficient dynamic and distributed rsa accumulator. [arXiv:0905.1307](https://arxiv.org/abs/0905.1307) (2009)
33. Wood, G.: Ethereum: A secure decentralised generalised transaction ledger. *Ethereum Project Yellow Paper* **151**, 1–32 (2014)
34. Liu, Z., Li, T., Li, P., Jia, C., Li, J.: Verifiable searchable encryption with aggregate keys for data sharing system. *Futur. Gener. Comput. Syst.* **78**, 778–788 (2018)
35. Kamara, S., Papamanthou, C., Roeder, T.: Cs2: A searchable cryptographic cloud storage system (2011)
36. New york times database: <https://developer.nytimes.com/docs> (2020)

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.